

Automated Data Annotation Strategy

Tyler Venner

January 12, 2026

1 Executive Summary

To train a robust YOLOv8 object detector on our dataset of approximately 50,000 high-resolution arthropod images, we require bounding box annotations. Manual annotation of a dataset of this magnitude is estimated to require over 500 human-hours.

We propose an **Automated Labeling Pipeline** utilizing *Zero-Shot Object Detection*. We will leverage a heavy-weight Foundation Model (Grounding DINO) to act as a “Teacher,” generating labels which will then be used to train a lightweight “Student” model (YOLOv8) for deployment.

2 Theoretical Framework: Knowledge Distillation

The core theoretical concept driving this strategy is **Knowledge Distillation** (specifically, Label Distillation). We perform a transfer of inductive biases from a large, open-vocabulary model to a closed-vocabulary, real-time model.

2.1 The Teacher: Grounding DINO

Grounding DINO is an open-set object detector that fuses text and image modalities. Unlike traditional detectors trained on a fixed set of classes (e.g., COCO’s 80 classes), Grounding DINO learns a joint embedding space for language and vision.

Given an image I and a text prompt T (e.g., “insect”), the model predicts bounding boxes B by maximizing the similarity between the image region features and the text embedding:

$$B_{pred} = \operatorname{argmax}_B P(B|I, T) \quad (1)$$

This allows us to label classes the model has never explicitly been trained on, provided it understands the semantic concept of the class.

Note on Label Noise: We acknowledge that teacher-generated labels are inherently noisy compared to human verification. However, empirical evidence in semi-supervised learning suggests that YOLO-style detectors are robust to moderate label noise when trained at scale, as the network tends to learn the dominant signal over idiosyncratic errors.

2.2 The Student: YOLOv8

YOLO (You Only Look Once) is optimized for inference speed. It requires fixed class definitions and cannot process text prompts. By training YOLO on the output of Grounding DINO, we effectively “compile” the massive knowledge of the foundation model into a compact numerical representation suitable for mobile deployment.

2.3 Handling Class Imbalance

Given the disparate species counts in our dataset (e.g., 50k Beetles vs. 600 Ticks), simple random sampling during training will lead to bias. During the YOLO training phase, class imbalance will be mitigated via oversampling of minority classes, focal loss to down-weight easy examples, or class-weighted loss functions as needed.

3 Implementation Architecture on Azure

To optimize for cost and throughput, we utilize the “Unzip-on-Compute” pattern rather than streaming small files from cloud storage.

3.1 Infrastructure Components

- **Storage Layer:** Azure Blob Storage (`raw-data-zips`). Contains the 12 archival ZIP files (approx. 60GB).
- **Compute Layer:** Azure Machine Learning Compute Instance (`Standard_NC4as_T4_v3`). Equipped with 4 vCPUs, 28GB RAM, and 1x NVIDIA Tesla T4 GPU.
- **Orchestration:** Python script utilizing the `autodistill` library.

3.2 Data Flow Pipeline

1. **Ingestion:** The Compute Instance downloads the target ZIP file (e.g., `BugLiteAI_spiders.zip`) from the Blob Container to its high-speed local NVMe SSD.
2. **Extraction:** The archive is unzipped locally. This avoids the high latency of listing thousands of blobs over the network.
3. **Inference (The Labeling Loop):**
 - The Grounding DINO model is loaded onto the T4 GPU.
 - **Ontology Definition:** We map visual concepts to a single detection class. For detection purposes, visually similar arthropods (e.g., spiders, scorpions) are mapped to a single coarse ‘insect’ or ‘arthropod’ class to improve recall.
 - **Refined Prompts:** Given the diversity of our dataset, a single prompt may struggle. We will apply class-specific prompts based on the folder name:
 - Spiders/Ticks/Bugs: `{"insect": "insect"}` (General)
 - Scorpions: `{"scorpion": "insect"}` (Distinct shape)
 - Caterpillars: `{"caterpillar": "insect"}` (Distinct texture)
 - The model iterates through the local images, predicting bounding boxes.
 - **Confidence Filtering:** Predictions with confidence scores < 0.35 are discarded to minimize false positives.
4. **Serialization:** The accepted bounding boxes are written to text files in the standard YOLO format:

`<class_id> <x_center> <y_center> <width> <height>`

where coordinates are normalized to $[0, 1]$.
5. **Egress:** The resulting folder of text files is zipped and uploaded to a new Azure Blob Container: `labeled-data-yolo`.

4 Pilot Phase Strategy

Before executing the full pipeline on 50,000 images, we will execute a **Pilot Test** to validate the infrastructure and estimate costs.

- **Sampling Rate (α):** We will set a sampling parameter $\alpha = 0.05$ (5%), processing approximately 2,500 images.
- **Verification:** The output labels will be manually inspected to ensure coordinate normalization is correct and the “Unzip-on-Compute” logic functions as expected.
- **Cost Projection:** We will measure the pilot duration (T_{pilot}) to extrapolate the full cost:

$$Cost_{total} \approx \frac{T_{pilot}}{\alpha} \times \text{Hourly Rate}_{\text{NC4as.T4}} \quad (2)$$

5 Human-in-the-Loop Verification

Automated labeling is not infallible. To ensure quality control without reviewing all 50,000 images, we apply **Stratified Sampling**.

1. Randomly sample $N = 100$ images from each of the 12 classes after auto-labeling.
2. Load these samples into a verification tool (e.g., LabelImg or Roboflow).
3. **Metric:** Calculate the IoU (Intersection over Union) of the auto-labels against a human ground truth for this subset.
4. If the error rate for a specific class (e.g., Phasmids/Walking Sticks) is high, we refine the text prompt for that specific class (e.g., changing "insect" to "stick insect") and re-run the inference.

6 Conclusion

This architecture minimizes cloud costs by utilizing transient compute for heavy lifting while storing only compressed archives. It leverages state-of-the-art multimodal AI to reduce the labeling burden by an estimated 95%, allowing the team to focus on model architecture and deployment rather than manual annotation.